

DAY 28

Ensemble Learning & Random Forests

60-Day Data Science Learning Journey | Muhammad Ahmad

Learning objectives

By the end of this guide, you should understand ensemble learning, bagging, boosting, voting, Random Forest internals, key hyperparameters, classification metrics, feature importance, and how to build and evaluate a complete Scikit-learn project.

1. What is Ensemble Learning?

Ensemble learning combines predictions from multiple models. The idea is that different models may make different mistakes, so combining their outputs can improve reliability. Improvement is not guaranteed: models need useful predictive skill, and identical errors are difficult to cancel out.

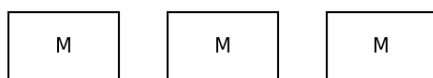
2. Main ensemble approaches

Bagging: train models independently on bootstrap samples and aggregate predictions. It often reduces variance. Random Forest is a bagging-based approach with additional random feature selection.

Boosting: train models sequentially, with later stages improving the overall learner by focusing on errors or difficult examples. Examples include AdaBoost, Gradient Boosting, and XGBoost.

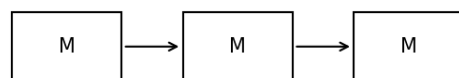
Voting: combine predictions from different models. Hard voting chooses the majority class, while soft voting combines class probabilities. Regression ensembles often average numerical predictions.

BAGGING



Independent training

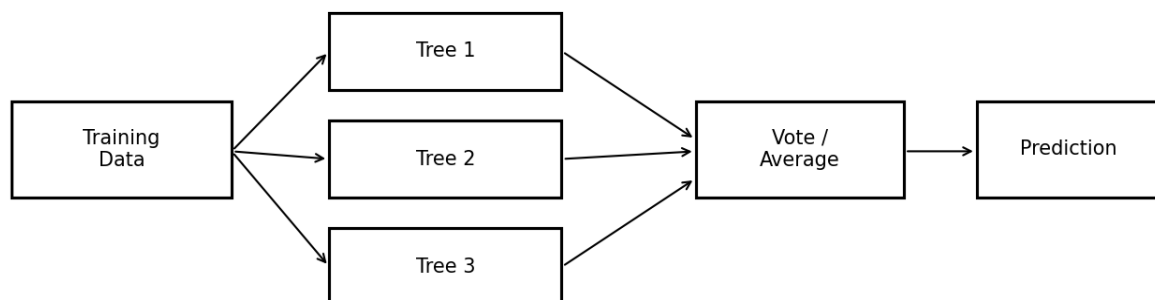
BOOSTING



Sequential improvement

3. What is a Random Forest?

A Random Forest combines many Decision Trees. Each tree uses a bootstrap sample of training rows, and each split considers a random subset of features. The trees are therefore not identical. Classification usually combines tree votes or class probabilities. Regression generally averages numerical tree predictions.



4. Random Forest workflow

1. Start with training data.
2. Draw bootstrap samples by sampling rows with replacement.
3. Train a Decision Tree on each sample.

4. At each split, consider a random subset of features.
5. Ask every tree for a prediction on a new row.
6. Aggregate the tree predictions using voting or averaging.

5. Why can it work well?

A single tree can be sensitive to small data changes. A forest averages or votes across trees trained with different samples and feature subsets. When the trees are reasonably accurate and make different errors, aggregation can reduce variance. More trees often stabilize results but increase memory use and prediction time.

6. Decision Tree vs Random Forest

Aspect	Decision Tree	Random Forest
Models	One tree	Many trees
Interpretability	Usually easier	More complex overall
Overfitting	Deep trees can overfit	Often reduces variance
Speed	Often faster	Can be slower
Scaling	Usually unnecessary	Usually unnecessary
Performance	Depends on data	Often strong, not guaranteed best

7. Key hyperparameters

n_estimators: number of trees. More trees may stabilize results but use more resources.

max_depth: maximum depth of each tree. Smaller depth limits complexity.

min_samples_split: minimum samples required to split an internal node.

min_samples_leaf: minimum samples allowed in a leaf.

max_features: number or fraction of features considered at each split.

class_weight: can help with class imbalance, but does not replace suitable metrics.

random_state: supports reproducibility for a fixed environment and configuration.

8. Do trees need feature scaling?

Decision Trees and Random Forests generally do not require StandardScaler or MinMaxScaler. They use threshold-based splits, and scaling usually preserves the ordering of values. Data quality and correct evaluation are much more important here.

9. Complete classification project

We will predict a student's Pass/Fail label from study hours, attendance, previous GPA, and assignment score. This is a small synthetic educational example, not a real-world validated student assessment system.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = {
    "study_hours": [1,2,2,3,3,4,5,5,6,6,7,7,8,8,9,9,10,10,11,12],
    "attendance": [50,55,60,62,65,68,72,75,78,80,82,85,87,90,91,93,95,96,98,99],
    "previous_gpa": [1.8,2.0,2.1,2.2,2.3,2.4,2.6,2.7,2.8,2.9,3.0,3.0,3.1,3.2,3.3,3.4,3.5,3.6,3.7,3.9],
    "assignment_score": [35,40,42,45,48,52,58,62,65,68,70,74,76,80,82,85,88,92,95,98],
    "passed": ["Fail","Fail","Fail","Fail","Fail","Fail","Fail","Pass","Pass","Pass","Pass",
               "Pass","Pass","Pass","Pass","Pass","Pass","Pass","Pass","Pass","Pass"]
}
df = pd.DataFrame(data)
print(df.head())

feature_names = ["study_hours", "attendance", "previous_gpa", "assignment_score"]
X = df[feature_names]
y = df["passed"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

forest = RandomForestClassifier(
    n_estimators=100, max_depth=4, min_samples_split=2,
    min_samples_leaf=1, max_features="sqrt", random_state=42
)
forest.fit(X_train, y_train)
y_pred = forest.predict(X_test)

print("Random Forest accuracy:", round(accuracy_score(y_test, y_pred), 3))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred, labels=forest.classes_))
print("Classification report:")
print(classification_report(y_test, y_pred, zero_division=0))

# Baseline comparison using the same train/test split
tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_train)
tree_pred = tree.predict(X_test)
print("Decision Tree accuracy:", round(accuracy_score(y_test, tree_pred), 3))

importance = pd.Series(forest.feature_importances_, index=feature_names).sort_values()
print("Feature importance:")
print(importance)
importance.plot(kind="barh")
plt.title("Random Forest Feature Importance")
plt.xlabel("Impurity-based importance")
plt.tight_layout()
plt.show()

new_student = pd.DataFrame({
    "study_hours": [6], "attendance": [82],
    "previous_gpa": [3.0], "assignment_score": [72]
})
print("New student prediction:", forest.predict(new_student)[0])
print("Class order:", forest.classes_)
print("Estimated probabilities:", forest.predict_proba(new_student)[0])
```

10. Code walkthrough, step by step

Imports: Pandas handles tables; Matplotlib creates the chart; train_test_split separates training and test rows; RandomForestClassifier trains the forest; DecisionTreeClassifier provides a baseline; metrics evaluate predictions.

Dataset: each row is one fictional student. The first four columns are features, and passed is the target label.

X and y: X contains model inputs. y contains the labels the model learns to predict.

Train/test split: test_size=0.25 holds out 25% of rows. random_state=42 makes the split repeatable. stratify=y tries to preserve the class proportions.

Model settings: n_estimators=100 trains 100 trees. max_depth=4 limits each tree. min_samples_split and min_samples_leaf control splitting and leaf size. max_features='sqrt' makes each split consider a random feature subset for classification.

fit: learns from training rows. **predict:** produces class labels. **accuracy_score:** measures the fraction correct. **confusion_matrix:** counts actual vs predicted labels in the specified class order. **classification_report:** prints precision, recall, F1 and support.

Baseline: a single Decision Tree is trained on the same split. The test set is tiny, so differences in score are unstable and should not

be generalized.

Feature importance: values summarize impurity reduction, not causation. Correlated features can share or distort importance.

New student: use the same feature names as during training. `predict_proba` gives estimated probabilities in the order shown by `forest.classes_`.

11. Evaluation metrics

Accuracy = correct predictions / total predictions.

Precision = $TP / (TP + FP)$. Of records predicted positive, how many were truly positive?

Recall = $TP / (TP + FN)$. Of all truly positive records, how many did the model find?

F1-score is the harmonic mean of precision and recall. Use the confusion matrix and understand which class is considered positive. Accuracy alone can be misleading on imbalanced data.

12. Bagging and out-of-bag evaluation

Bootstrap samples draw rows with replacement. Some rows can appear more than once in a tree's sample, while others are left out. With `bootstrap=True` and `oob_score=True`, a Random Forest can estimate an out-of-bag score using left-out rows for each tree. This estimate can be noisy on small datasets and does not eliminate the value of a held-out test set or cross-validation.

```
from sklearn.ensemble import RandomForestClassifier

oob_forest = RandomForestClassifier(
    n_estimators=300,
    bootstrap=True,
    oob_score=True,
    random_state=42
)
oob_forest.fit(X_train, y_train)
print("Out-of-bag score:", oob_forest.oob_score_)
```

13. Practice tasks with solution guidance

- 1. Use 50 trees:** set `n_estimators=50`, refit and compare held-out metrics.
- 2. Use 200 trees:** set `n_estimators=200`. More trees may stabilize results but do not guarantee higher accuracy.
- 3. Compare depth:** try `max_depth=2` and `max_depth=8`, and compare test metrics.
- 4. Confusion matrix:** print `confusion_matrix(y_test, y_pred, labels=forest.classes_)`.
- 5. Classification report:** print `classification_report(y_test, y_pred, zero_division=0)`.
- 6. Compare models:** train a Decision Tree and Random Forest on the same split, then compare accuracy, precision, recall, and F1.
- 7. Plot importance:** create a Series from `forest.feature_importances_`, indexed by `feature_names`, sort it, and plot a horizontal bar chart.
- 8. New record:** create a one-row DataFrame with all four feature columns and call `forest.predict(new_student)`.
- 9. Change max_features:** compare 'sqrt' with 1.0 and record the effect.
- 10. Regression:** use `RandomForestRegressor` with a numeric target such as `exam_marks`, then evaluate MAE, MSE, and R².

14. Random Forest Regression

For numeric targets, use `RandomForestRegressor`. This example assumes `X_train`, `X_test`, `y_train`, and `y_test` are based on a numeric target such as `exam_marks`. Do not pass the Pass/Fail text target to a regressor.

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

regressor = RandomForestRegressor(
    n_estimators=100, max_depth=5, random_state=42
)
regressor.fit(X_train, y_train) # y_train must be numeric
reg_pred = regressor.predict(X_test)

print("MAE:", mean_absolute_error(y_test, reg_pred))
print("MSE:", mean_squared_error(y_test, reg_pred))
print("R2:", r2_score(y_test, reg_pred))
```

15. Common mistakes

- Evaluating only on training data.
- Assuming more trees always improve results.
- Providing different feature columns at prediction time.
- Treating feature importance as causal evidence.
- Relying on accuracy alone for imbalanced classes.
- Allowing target information to leak into features.
- Comparing models on different data splits.

16. Summary and next step

Ensemble learning combines models. Bagging trains models independently on bootstrap samples. Boosting trains models sequentially. Random Forest combines randomized Decision Trees and can solve classification or regression. It often reduces variance and usually does not require feature scaling. Always evaluate with held-out data and interpret metrics in context. Next, study boosting methods such as AdaBoost and Gradient Boosting.